LAB PROGRAMS UNIT-4

NAME: RAJASHREE S

ROLL NO: 192521079

COURSE: CRYPTOGRAPHY AND NETWORK SECURITY

CODE: CSA-5104

SLOT: C

IPSec

Program 1: IPSec Architecture Simulation

Aim: Design and implement a simulation to demonstrate the IPSec architecture and explain the interaction between its components.

Requirements:

- Model the core IPSec components: Security Policy Database (SPD), Security Association Database (SAD), IKE (key management), and AH/ESP.
- Simulate an outbound packet flow through policy lookup, SA creation, and protection.
- Demonstrate PROTECT, BYPASS, and DISCARD policy outcomes.

Expected Learning Outcome: Understand how IPSec components interact to secure, bypass, or drop IP traffic.

```python
# Program 1: IPSec Architecture Simulation
# Demonstrates interaction between IPSec components: SPD, SAD, IKE, AH, ESP

class SPD:
    """Security Policy Database - decides what to do with a packet"""
    def __init__(self):
        self.policies = {
            ("10.0.0.5", "10.0.0.9"): "PROTECT-ESP",
            ("10.0.0.5", "10.0.0.20"): "BYPASS",
        }
    def lookup(self, src, dst):
        return self.policies.get((src, dst), "DISCARD")

class SAD:
    """Security Association Database"""
    def __init__(self):
        self.sas = {}
    def add_sa(self, spi, protocol, algo, key):
        self.sas[spi] = {"protocol": protocol, "algo": algo, "key": key}
    def get_sa(self, spi):
        return self.sas.get(spi)

class IKE:
    """Simplified key management engine"""
    def negotiate(self, peer):
        print(f"  [IKE] Negotiating session key with {peer} ... key established")
        return "NEGOTIATED-KEY-0xA1F3"

def process_outbound(spd, sad, ike, src, dst):
    print(f"\nOutbound packet: {src} -> {dst}")
    decision = spd.lookup(src, dst)
    print(f"  [SPD] Policy decision: {decision}")
    if decision == "DISCARD":
        print("  Packet dropped.")
        return
    if decision == "BYPASS":
        print("  Packet sent in cleartext.")
        return
    proto = decision.split("-")[1]
    key = ike.negotiate(dst)
    spi = 0x2001
    sad.add_sa(spi, proto, "AES-128" if proto == "ESP" else "HMAC-SHA256", key)
    sa = sad.get_sa(spi)
    print(f"  [SAD] SA created -> SPI={hex(spi)}, protocol={sa['protocol']},
        algo={sa['algo']}")
    print(f"  [{proto}] Packet protected using SA {hex(spi)} and forwarded to {dst}")

if __name__ == "__main__":
    spd, sad, ike = SPD(), SAD(), IKE()
    print("=== IPSec Architecture Simulation ===")
    print("Components: SPD (policy), SAD (associations), IKE (key mgmt), AH/ESP
        (protection)")
    process_outbound(spd, sad, ike, "10.0.0.5", "10.0.0.9")
    process_outbound(spd, sad, ike, "10.0.0.5", "10.0.0.20")
    process_outbound(spd, sad, ike, "10.0.0.5", "192.168.9.9")
```

```
=== IPSec Architecture Simulation ===
Components: SPD (policy), SAD (associations), IKE (key mgmt), AH/ESP (protection)

Outbound packet: 10.0.0.5 -> 10.0.0.9
  [SPD] Policy decision: PROTECT-ESP
  [IKE] Negotiating session key with 10.0.0.9 ... key established
  [SAD] SA created -> SPI=0x2001, protocol=ESP, algo=AES-128
  [ESP] Packet protected using SA 0x2001 and forwarded to 10.0.0.9

Outbound packet: 10.0.0.5 -> 10.0.0.20
  [SPD] Policy decision: BYPASS
  Packet sent in cleartext.

Outbound packet: 10.0.0.5 -> 192.168.9.9
  [SPD] Policy decision: DISCARD
  Packet dropped.
```

Program 2: Security Association (SA) Database and SA Lookup

Aim: Develop a program to create a Security Association (SA) database and simulate SA lookup for incoming IP packets.

Requirements:

- Maintain an SA database indexed by SPI (Security Parameter Index).
- Store protocol, algorithm and key for each SA.
- Look up the correct SA for incoming packets and handle the no-match case.

Expected Learning Outcome: Understand how SAs are stored and retrieved to process IPSec traffic.

```python
# Program 2: Security Association (SA) Database and SA Lookup
sa_database = {
    0x1001: {"src": "192.168.1.10", "dst": "192.168.1.20", "protocol": "ESP", "algo":
        "AES-128", "key": "K1a2B3"},
    0x1002: {"src": "192.168.1.10", "dst": "192.168.1.30", "protocol": "AH",  "algo": "HMAC-
        SHA256", "key": "K9x8Y7"},
    0x1003: {"src": "192.168.1.11", "dst": "192.168.1.20", "protocol": "ESP", "algo":
        "AES-256", "key": "K5m6N7"},
}

def sa_lookup(spi):
    return sa_database.get(spi)

incoming_packets = [
    {"spi": 0x1001, "src": "192.168.1.10", "dst": "192.168.1.20"},
    {"spi": 0x1002, "src": "192.168.1.10", "dst": "192.168.1.30"},
    {"spi": 0x9999, "src": "10.0.0.1", "dst": "10.0.0.2"},
]

print("=== SA Database ===")
for spi, sa in sa_database.items():
    print(f"SPI={hex(spi)}  {sa}")

print("\n=== Incoming Packet SA Lookup ===")
for pkt in incoming_packets:
    sa = sa_lookup(pkt["spi"])
    print(f"\nPacket {pkt['src']} -> {pkt['dst']}, SPI={hex(pkt['spi'])}")
    if sa:
        print(f"  SA found: protocol={sa['protocol']}, algo={sa['algo']}, key={sa['key']}")
        print("  Action: Process packet using this SA")
    else:
        print("  No matching SA found -> Packet DROPPED")
```

```
=== SA Database ===
SPI=0x1001  {'src': '192.168.1.10', 'dst': '192.168.1.20', 'protocol': 'ESP', 'algo':
    'AES-128', 'key': 'K1a2B3'}
SPI=0x1002  {'src': '192.168.1.10', 'dst': '192.168.1.30', 'protocol': 'AH', 'algo': 'HMAC-
    SHA256', 'key': 'K9x8Y7'}
SPI=0x1003  {'src': '192.168.1.11', 'dst': '192.168.1.20', 'protocol': 'ESP', 'algo':
    'AES-256', 'key': 'K5m6N7'}

=== Incoming Packet SA Lookup ===

Packet 192.168.1.10 -> 192.168.1.20, SPI=0x1001
  SA found: protocol=ESP, algo=AES-128, key=K1a2B3
  Action: Process packet using this SA

Packet 192.168.1.10 -> 192.168.1.30, SPI=0x1002
  SA found: protocol=AH, algo=HMAC-SHA256, key=K9x8Y7
  Action: Process packet using this SA

Packet 10.0.0.1 -> 10.0.0.2, SPI=0x9999
  No matching SA found -> Packet DROPPED
```

Program 3: Authentication Header (AH) Protocol

Aim: Implement the Authentication Header (AH) protocol to provide integrity and origin authentication for IP packets.

Requirements:

- Compute an ICV over the immutable packet fields using HMAC.
- Attach an AH header containing SPI, sequence number and ICV.
- Verify the ICV at the receiver and detect tampering.

Expected Learning Outcome: Understand how AH guarantees integrity and origin authentication without confidentiality.

```python
# Program 3: Authentication Header (AH) Protocol
import hmac, hashlib

def compute_icv(key, data):
    return hmac.new(key, data, hashlib.sha256).hexdigest()

def build_ah_packet(src, dst, payload, key, spi=0x3001, seq=1):
    immutable_fields = f"{src}|{dst}|{payload}".encode()
    icv = compute_icv(key, immutable_fields)
    ah_header = {"next_header": "TCP", "spi": spi, "seq": seq, "icv": icv}
    return {"ip_header": (src, dst), "ah_header": ah_header, "payload": payload}

def verify_ah_packet(packet, key):
    src, dst = packet["ip_header"]
    immutable_fields = f"{src}|{dst}|{packet['payload']}".encode()
    expected_icv = compute_icv(key, immutable_fields)
    return expected_icv == packet["ah_header"]["icv"]

key = b"shared-secret-key"
pkt = build_ah_packet("172.16.0.1", "172.16.0.2", "HELLO-SERVER", key)
print("=== AH Protocol Simulation ===")
print("Constructed packet:", pkt)

print("\nReceiver verifying integrity & origin authentication...")
print("Result:", "VALID (integrity + authenticity confirmed)" if verify_ah_packet(pkt, key)
    else "INVALID")

print("\nTampering payload in transit...")
pkt["payload"] = "HELLO-ATTACKER"
print("Result:", "VALID" if verify_ah_packet(pkt, key) else "INVALID (tampering detected,
    packet rejected)")
```

```
=== AH Protocol Simulation ===
Constructed packet: {'ip_header': ('172.16.0.1', '172.16.0.2'), 'ah_header': {'next_header':
    'TCP', 'spi': 12289, 'seq': 1, 'icv':
    '0425f7b7c87be9da89f886f48cd949e76c666ad09ffceb60c7d25a455042b653'}, 'payload': 'HELLO-
    SERVER'}

Receiver verifying integrity & origin authentication...
Result: VALID (integrity + authenticity confirmed)

Tampering payload in transit...
Result: INVALID (tampering detected, packet rejected)
```

Program 4: Integrity Check Value (ICV) Calculation

Aim: Write a program to calculate the Integrity Check Value (ICV) used in the Authentication Header.

Requirements:

- Compute ICV using HMAC with multiple hash algorithms (SHA-256, SHA-1, MD5).
- Recompute the ICV at the receiver to verify a match.

Expected Learning Outcome: Understand how ICV is derived and used to confirm packet integrity.

```python
# Program 4: Integrity Check Value (ICV) Calculation for AH
import hmac, hashlib

def calculate_icv(data: bytes, key: bytes, algo="sha256"):
    return hmac.new(key, data, getattr(hashlib, algo)).hexdigest()

key = b"AH-Secret-Key-001"
packet_data = b"SRC=10.1.1.1 DST=10.1.1.9 SEQ=45 PAYLOAD=BankTransferRequest"

print("=== ICV Calculation for Authentication Header ===")
print("Packet data:", packet_data.decode())
print("Key        :", key.decode())

icv_sha256 = calculate_icv(packet_data, key, "sha256")
icv_sha1 = calculate_icv(packet_data, key, "sha1")
icv_md5 = calculate_icv(packet_data, key, "md5")

print(f"\nICV (HMAC-SHA256): {icv_sha256}")
print(f"ICV (HMAC-SHA1)  : {icv_sha1}")
print(f"ICV (HMAC-MD5)   : {icv_md5}")

print("\nRecomputing ICV on receiver side to verify...")
recv_icv = calculate_icv(packet_data, key, "sha256")
print("Match:", recv_icv == icv_sha256)
```

```
=== ICV Calculation for Authentication Header ===
Packet data: SRC=10.1.1.1 DST=10.1.1.9 SEQ=45 PAYLOAD=BankTransferRequest
Key        : AH-Secret-Key-001

ICV (HMAC-SHA256): f4ca757ce5525a92983e769026e53bf1ebfd22be5ab1c6c1f7c31f11602f1627
ICV (HMAC-SHA1)  : ffd2977a2c052b26755e3be65105a1ac924e1dba
ICV (HMAC-MD5)   : 9bbeb4da36b0b930d7a2d2cd6f54c31b

Recomputing ICV on receiver side to verify...
Match: True
```

Program 5: Packet Transmission With and Without AH

Aim: Develop a simulation that compares packet transmission with and without the Authentication Header.

Requirements:

- Simulate a tampering attack on a packet sent without AH.
- Simulate the same attack on a packet protected by AH.
- Show that AH detects and rejects the tampered packet.

Expected Learning Outcome: Observe the practical value of AH in detecting in-transit tampering.

```python
# Program 5: Packet Transmission With vs Without AH
import hmac, hashlib, random

def send_without_ah(payload):
    print(f"Sent (no AH): '{payload}'")
    # Simulate attacker tampering during transit
    tampered = payload.replace("1000", "9000")
    print(f"Received     : '{tampered}'")
    print("No way to detect tampering -> Receiver silently accepts forged data!")

def send_with_ah(payload, key):
    icv = hmac.new(key, payload.encode(), hashlib.sha256).hexdigest()
    print(f"Sent (with AH): '{payload}'  ICV={icv[:16]}...")
    tampered = payload.replace("1000", "9000")
    recv_icv = hmac.new(key, tampered.encode(), hashlib.sha256).hexdigest()
    print(f"Received      : '{tampered}'")
    if recv_icv != icv:
        print("AH verification FAILED -> tampering detected, packet DISCARDED")
    else:
        print("AH verification passed")

key = b"integrity-key"
print("=== Without AH ===")
send_without_ah("TransferAmount=1000")
print("\n=== With AH ===")
send_with_ah("TransferAmount=1000", key)
```

```
=== Without AH ===
Sent (no AH): 'TransferAmount=1000'
Received     : 'TransferAmount=9000'
No way to detect tampering -> Receiver silently accepts forged data!

=== With AH ===
Sent (with AH): 'TransferAmount=1000'  ICV=b6028bca2f8252ce...
Received      : 'TransferAmount=9000'
AH verification FAILED -> tampering detected, packet DISCARDED
```

Program 6: Encapsulating Security Payload (ESP) Protocol

Aim: Implement the Encapsulating Security Payload (ESP) protocol to encrypt and authenticate IP packets.

Requirements:

- Encrypt the payload using AES-CBC with a random IV.
- Compute an HMAC-based ICV over the ciphertext for authentication.
- Decrypt and verify the ESP packet at the receiver.

Expected Learning Outcome: Understand how ESP provides both confidentiality and authentication.

```python
# Program 6: Encapsulating Security Payload (ESP) Protocol
import os, hmac, hashlib
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.primitives import padding as sympad

def esp_encrypt(plaintext, enc_key, auth_key):
    iv = os.urandom(16)
    padder = sympad.PKCS7(128).padder()
    padded = padder.update(plaintext.encode()) + padder.finalize()
    cipher = Cipher(algorithms.AES(enc_key), modes.CBC(iv))
    ciphertext = cipher.encryptor().update(padded) + cipher.encryptor().finalize()
    icv = hmac.new(auth_key, iv + ciphertext, hashlib.sha256).hexdigest()
    return {"iv": iv, "ciphertext": ciphertext, "icv": icv}

def esp_decrypt(esp_pkt, enc_key, auth_key):
    expected_icv = hmac.new(auth_key, esp_pkt["iv"] + esp_pkt["ciphertext"],
        hashlib.sha256).hexdigest()
    if expected_icv != esp_pkt["icv"]:
        return None, "AUTH FAILED"
    cipher = Cipher(algorithms.AES(enc_key), modes.CBC(esp_pkt["iv"]))
    padded = cipher.decryptor().update(esp_pkt["ciphertext"]) +
        cipher.decryptor().finalize()
    unpadder = sympad.PKCS7(128).unpadder()
    plaintext = unpadder.update(padded) + unpadder.finalize()
    return plaintext.decode(), "OK"

enc_key = os.urandom(16)
auth_key = os.urandom(32)
message = "CONFIDENTIAL: Quarterly report attached."

print("=== ESP Protocol Simulation ===")
print("Original message:", message)
esp_pkt = esp_encrypt(message, enc_key, auth_key)
print(f"ESP packet -> IV={esp_pkt['iv'].hex()[:16]}...
    CIPHERTEXT={esp_pkt['ciphertext'].hex()[:32]}...  ICV={esp_pkt['icv'][:16]}...")

plaintext, status = esp_decrypt(esp_pkt, enc_key, auth_key)
print(f"\nDecryption status: {status}")
print("Recovered message:", plaintext)
```

```
=== ESP Protocol Simulation ===
Original message: CONFIDENTIAL: Quarterly report attached.
ESP packet -> IV=5a4f649231838923...  CIPHERTEXT=754eec8fca0c292832bef1a2fcdd61c7...
    ICV=f98a8b803f66004f...

Decryption status: OK
Recovered message: CONFIDENTIAL: Quarterly report attached.
```

Program 7: IPSec Transport Mode and Tunnel Mode

Aim: Create a program to demonstrate IPSec transport mode and tunnel mode using sample packets.

Requirements:

- Show transport mode packet structure (original IP header retained).

- Show tunnel mode packet structure (entire packet encapsulated under a new outer IP header).
- Highlight the structural difference between the two modes.

Expected Learning Outcome: Understand when to use transport mode (host-to-host) versus tunnel mode (gateway-to-gateway/VPN).

```
# Program 7: IPSec Transport Mode vs Tunnel Mode
def transport_mode(orig_ip_header, tcp_payload):
    print("TRANSPORT MODE (host-to-host):")
    print(f"  [{orig_ip_header}][ESP/AH Header][{tcp_payload}][ESP Trailer]")
    print("  Original IP header is reused; only the payload is protected.")

def tunnel_mode(orig_ip_header, tcp_payload, gateway_ip_header):
    print("TUNNEL MODE (gateway-to-gateway / VPN):")
    print(f"  [{gateway_ip_header}][ESP/AH Header][{orig_ip_header}][{tcp_payload}][ESP
        Trailer]")
    print("  Entire original packet (incl. its IP header) is encapsulated inside a new outer
        IP header.")

sample_ip_header = "IP(src=192.168.1.5, dst=192.168.1.9)"
sample_payload = "TCP+DATA"
gateway_header = "IP(src=GW-A=203.0.113.1, dst=GW-B=198.51.100.1)"

print("=== IPSec Modes Demonstration ===\n")
transport_mode(sample_ip_header, sample_payload)
print()
tunnel_mode(sample_ip_header, sample_payload, gateway_header)
```

```
=== IPSec Modes Demonstration ===

TRANSPORT MODE (host-to-host):
  [IP(src=192.168.1.5, dst=192.168.1.9)][ESP/AH Header][TCP+DATA][ESP Trailer]
  Original IP header is reused; only the payload is protected.

TUNNEL MODE (gateway-to-gateway / VPN):
  [IP(src=GW-A=203.0.113.1, dst=GW-B=198.51.100.1)][ESP/AH Header][IP(src=192.168.1.5,
      dst=192.168.1.9)][TCP+DATA][ESP Trailer]
  Entire original packet (incl. its IP header) is encapsulated inside a new outer IP header.
```

Program 8: ESP Encapsulation and Decapsulation

Aim: Simulate the encapsulation and decapsulation process of ESP for secure communication.

Requirements:

- Encapsulate a payload with ESP header, IV, ciphertext and trailer.
- Decapsulate at the receiver to recover the original payload.
- Verify the recovered payload matches the original.

Expected Learning Outcome: Understand the step-by-step ESP encapsulation/decapsulation pipeline.

```python
# Program 8: ESP Encapsulation and Decapsulation
import os
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.primitives import padding as sympad

def esp_encapsulate(payload, key, spi=0x4001, seq=1):
    iv = os.urandom(16)
    padder = sympad.PKCS7(128).padder()
    padded = padder.update(payload.encode() + b"\x02\x02") + padder.finalize()  # pad len +
        next-hdr as trailer sim
    cipher = Cipher(algorithms.AES(key), modes.CBC(iv))
    ct = cipher.encryptor().update(padded) + cipher.encryptor().finalize()
    return {"spi": spi, "seq": seq, "iv": iv, "ciphertext": ct}

def esp_decapsulate(pkt, key):
    cipher = Cipher(algorithms.AES(key), modes.CBC(pkt["iv"]))
    padded = cipher.decryptor().update(pkt["ciphertext"]) + cipher.decryptor().finalize()
    unpadder = sympad.PKCS7(128).unpadder()
    data = unpadder.update(padded) + unpadder.finalize()
    return data[:-2].decode()  # strip simulated ESP trailer

key = os.urandom(16)
payload = "GET /secure/report.pdf HTTP/1.1"

print("=== ESP Encapsulation ===")
esp_pkt = esp_encapsulate(payload, key)
print(f"SPI={hex(esp_pkt['spi'])} SEQ={esp_pkt['seq']} IV={esp_pkt['iv'].hex()[:16]}...")
print(f"Encrypted ESP payload: {esp_pkt['ciphertext'].hex()[:40]}...")

print("\n=== ESP Decapsulation ===")
recovered = esp_decapsulate(esp_pkt, key)
print("Recovered original payload:", recovered)
```

```
=== ESP Encapsulation ===
SPI=0x4001 SEQ=1 IV=8ff6d49548e07154...
Encrypted ESP payload: 024abf1916881779d45a7a6d4815f3f58d68d6a1...

=== ESP Decapsulation ===
Recovered original payload: GET /secure/report.pdf HTTP/1.1
```

Program 9: Combining AH and ESP

Aim: Design a program that combines AH and ESP to provide confidentiality, integrity, and authentication.

Requirements:

- Apply ESP first to encrypt the payload (confidentiality).
- Apply AH over the resulting packet for integrity and origin authentication.
- Explain the resulting layered protection.

Expected Learning Outcome: Understand how AH and ESP can be combined for complete protection.

```python
# Program 9: Combining AH and ESP for Confidentiality, Integrity, and Authentication
import os, hmac, hashlib
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.primitives import padding as sympad

def esp_protect(payload, enc_key):
    iv = os.urandom(16)
    padder = sympad.PKCS7(128).padder()
    padded = padder.update(payload.encode()) + padder.finalize()
    cipher = Cipher(algorithms.AES(enc_key), modes.CBC(iv))
    ct = cipher.encryptor().update(padded) + cipher.encryptor().finalize()
    return iv, ct

def ah_protect(src, dst, esp_iv, esp_ct, ah_key):
    data = src.encode() + dst.encode() + esp_iv + esp_ct
    return hmac.new(ah_key, data, hashlib.sha256).hexdigest()

enc_key, ah_key = os.urandom(16), os.urandom(32)
src, dst = "10.10.10.1", "10.10.10.2"
message = "Transfer $50,000 to account 88213"

print("=== Combined AH + ESP Simulation ===")
print("Step 1: ESP provides confidentiality (encrypt payload)")
iv, ct = esp_protect(message, enc_key)
print(f"  ESP ciphertext: {ct.hex()[:32]}...")

print("Step 2: AH provides integrity + origin authentication (over IP header + ESP data)")
icv = ah_protect(src, dst, iv, ct, ah_key)
print(f"  AH ICV: {icv[:32]}...")

print("\nFinal packet: [IP][AH Header ICV][ESP Header][Encrypted Payload][ESP Trailer]")
print("=> Confidentiality (ESP) + Integrity & Authentication (AH) achieved together.")
```

```
=== Combined AH + ESP Simulation ===
Step 1: ESP provides confidentiality (encrypt payload)
  ESP ciphertext: a079c435e6a2592db8b38cb6a5b106c0...
Step 2: AH provides integrity + origin authentication (over IP header + ESP data)
  AH ICV: c6811169c51c9d907c1df09d8f7e8d2d...

Final packet: [IP][AH Header ICV][ESP Header][Encrypted Payload][ESP Trailer]
=> Confidentiality (ESP) + Integrity & Authentication (AH) achieved together.
```

Program 10: Multiple Security Associations for Different Sessions

Aim: Implement multiple Security Associations and demonstrate their usage for different communication sessions.

Requirements:

- Maintain an SA table with entries for multiple independent sessions.
- Select and apply the correct SA per session when sending data.
- Handle sessions with no configured SA.

Expected Learning Outcome: Understand how a gateway manages many concurrent SAs for different peers.

```python
# Program 10: Multiple Security Associations for Different Sessions
sa_table = [
    {"session": "Branch-A <-> HQ",   "spi": 0x5001, "protocol": "ESP", "algo": "AES-256",
        "key": "keyA"},
    {"session": "Branch-B <-> HQ",   "spi": 0x5002, "protocol": "ESP", "algo": "AES-128",
        "key": "keyB"},
    {"session": "Partner VPN link",  "spi": 0x5003, "protocol": "AH",  "algo": "HMAC-
        SHA256", "key": "keyC"},
]

def get_sa_for_session(session_name):
    for sa in sa_table:
        if sa["session"] == session_name:
            return sa
    return None

def send_on_session(session_name, data):
    sa = get_sa_for_session(session_name)
    print(f"\nSession: {session_name}")
    if not sa:
        print("  No SA configured, cannot send securely.")
        return
    print(f"  Using SA SPI={hex(sa['spi'])}, protocol={sa['protocol']}, algo={sa['algo']}")
    print(f"  Data '{data}' protected and transmitted.")

print("=== Multiple Simultaneous IPSec Sessions ===")
for sa in sa_table:
    print(sa)

send_on_session("Branch-A <-> HQ", "Sales report Q3")
send_on_session("Branch-B <-> HQ", "Inventory sync")
send_on_session("Partner VPN link", "Order confirmation #4471")
```

```
=== Multiple Simultaneous IPSec Sessions ===
{'session': 'Branch-A <-> HQ', 'spi': 20481, 'protocol': 'ESP', 'algo': 'AES-256', 'key':
    'keyA'}
{'session': 'Branch-B <-> HQ', 'spi': 20482, 'protocol': 'ESP', 'algo': 'AES-128', 'key':
    'keyB'}
{'session': 'Partner VPN link', 'spi': 20483, 'protocol': 'AH', 'algo': 'HMAC-SHA256',
    'key': 'keyC'}

Session: Branch-A <-> HQ
  Using SA SPI=0x5001, protocol=ESP, algo=AES-256
  Data 'Sales report Q3' protected and transmitted.

Session: Branch-B <-> HQ
  Using SA SPI=0x5002, protocol=ESP, algo=AES-128
  Data 'Inventory sync' protected and transmitted.

Session: Partner VPN link
  Using SA SPI=0x5003, protocol=AH, algo=HMAC-SHA256
  Data 'Order confirmation #4471' protected and transmitted.
```

Program 11: Manual Key Management Simulation

Aim: Develop a key management simulation using manual key configuration for IPSec communication.

Requirements:

- Allow an administrator to manually configure a pre-shared key between two peers.
- Establish SAs only where a key has been configured.
- Discuss the scalability limitation of manual keying.

Expected Learning Outcome: Understand manual keying and why automated key exchange (IKE) is preferred at scale.

```
# Program 11: Key Management Simulation using Manual Key Configuration
peers = {}

def configure_key(peer_a, peer_b, key):
    peers[(peer_a, peer_b)] = key
    peers[(peer_b, peer_a)] = key
    print(f"Admin manually configured pre-shared key between {peer_a} and {peer_b}: {key}")

def get_key(a, b):
    return peers.get((a, b))

print("=== Manual Key Configuration for IPSec ===")
configure_key("RouterA", "RouterB", "9F3C7A1E5B2D4F60")
configure_key("RouterA", "RouterC", "1122AABBCCDD9988")

print("\nEstablishing IPSec tunnels using manually configured keys:")
for a, b in [("RouterA", "RouterB"), ("RouterA", "RouterC"), ("RouterB", "RouterC")]:
    key = get_key(a, b)
    if key:
        print(f"  {a} <-> {b}: SA established using key {key}")
    else:
        print(f"  {a} <-> {b}: No manual key configured -> tunnel cannot be established")

print("\nLimitation: manual key config does not scale and keys must be re-entered if
    compromised (motivates IKE).")
```

```
=== Manual Key Configuration for IPSec ===
Admin manually configured pre-shared key between RouterA and RouterB: 9F3C7A1E5B2D4F60
Admin manually configured pre-shared key between RouterA and RouterC: 1122AABBCCDD9988

Establishing IPSec tunnels using manually configured keys:
  RouterA <-> RouterB: SA established using key 9F3C7A1E5B2D4F60
  RouterA <-> RouterC: SA established using key 1122AABBCCDD9988
  RouterB <-> RouterC: No manual key configured -> tunnel cannot be established

Limitation: manual key config does not scale and keys must be re-entered if compromised
    (motivates IKE).
```

Program 12: Simplified Internet Key Exchange (IKE) Protocol

Aim: Create a simplified Internet Key Exchange (IKE) protocol simulation for secure session establishment.

Requirements:

- Simulate IKE Phase 1 SA negotiation.
- Perform a Diffie-Hellman key exchange to derive a shared secret.
- Derive a session key and use it to establish an IPSec SA (Phase 2).

Expected Learning Outcome: Understand how IKE automates secure session key establishment for IPSec.

```python
# Program 12: Simplified Internet Key Exchange (IKE) Protocol Simulation
import random, hashlib

# Simplified Diffie-Hellman parameters (small numbers for demo only)
P = 23   # prime modulus
G = 5    # generator

def generate_private_key():
    return random.randint(2, P - 2)

def compute_public_key(private_key):
    return pow(G, private_key, P)

def compute_shared_secret(their_public, my_private):
    return pow(their_public, my_private, P)

print("=== Simplified IKE: Phase 1 (SA negotiation) ===")
print("Initiator and Responder agree on: encryption=AES-128, hash=SHA-256, DH group=demo-
    MODP")

print("\n=== Phase 1: Diffie-Hellman Key Exchange ===")
initiator_priv = generate_private_key()
responder_priv = generate_private_key()
initiator_pub = compute_public_key(initiator_priv)
responder_pub = compute_public_key(responder_priv)
print(f"Initiator public value sent:  {initiator_pub}")
print(f"Responder public value sent:  {responder_pub}")

initiator_secret = compute_shared_secret(responder_pub, initiator_priv)
responder_secret = compute_shared_secret(initiator_pub, responder_priv)
print(f"\nInitiator computed shared secret: {initiator_secret}")
print(f"Responder computed shared secret: {responder_secret}")
assert initiator_secret == responder_secret
session_key = hashlib.sha256(str(initiator_secret).encode()).hexdigest()
print(f"Derived session key (SKEYID): {session_key[:32]}...")

print("\n=== Phase 2: Quick Mode (IPSec SA established using derived key) ===")
print(f"IPSec SA created with SPI=0x6001 using session key {session_key[:16]}...")
```

```
=== Simplified IKE: Phase 1 (SA negotiation) ===
Initiator and Responder agree on: encryption=AES-128, hash=SHA-256, DH group=demo-MODP

=== Phase 1: Diffie-Hellman Key Exchange ===
Initiator public value sent:  17
Responder public value sent:  22

Initiator computed shared secret: 22
Responder computed shared secret: 22
Derived session key (SKEYID): 785f3ec7eb32f30b90cd0fcf3657d388...

=== Phase 2: Quick Mode (IPSec SA established using derived key) ===
IPSec SA created with SPI=0x6001 using session key 785f3ec7eb32f30b...
```

Program 13: Anti-Replay Protection using Sequence Numbers

Aim: Implement anti-replay protection using sequence numbers in IPSec communication.

Requirements:

- Maintain a sliding replay window keyed on sequence number.
- Accept new and in-window packets; reject replayed or too-old packets.
- Demonstrate detection of a replayed packet.

Expected Learning Outcome: Understand how the sliding-window mechanism prevents replay attacks.

```python
# Program 13: Anti-Replay Protection using Sequence Numbers
WINDOW_SIZE = 8

class AntiReplayWindow:
    def __init__(self, size=WINDOW_SIZE):
        self.size = size
        self.highest_seq = 0
        self.window = set()

    def check_and_update(self, seq):
        if seq > self.highest_seq:
            shift = seq - self.highest_seq
            self.window = {s + shift for s in self.window if s + shift < self.size}
            self.window.add(0)
            self.highest_seq = seq
            return "ACCEPTED (new highest sequence)"
        diff = self.highest_seq - seq
        if diff >= self.size:
            return "REJECTED (too old, outside window)"
        if diff in self.window:
            return "REJECTED (replay - already seen)"
        self.window.add(diff)
        return "ACCEPTED (within window, not seen before)"

arw = AntiReplayWindow()
incoming_sequence_numbers = [1, 2, 4, 3, 8, 4, 2, 15, 5]

print("=== Anti-Replay Protection Simulation (window size = 8) ===")
for seq in incoming_sequence_numbers:
    result = arw.check_and_update(seq)
    print(f"Packet SEQ={seq:<3} -> {result}")
```

```
=== Anti-Replay Protection Simulation (window size = 8) ===
Packet SEQ=1   -> ACCEPTED (new highest sequence)
Packet SEQ=2   -> ACCEPTED (new highest sequence)
Packet SEQ=4   -> ACCEPTED (new highest sequence)
Packet SEQ=3   -> ACCEPTED (within window, not seen before)
Packet SEQ=8   -> ACCEPTED (new highest sequence)
Packet SEQ=4   -> REJECTED (replay - already seen)
Packet SEQ=2   -> REJECTED (replay - already seen)
Packet SEQ=15  -> ACCEPTED (new highest sequence)
Packet SEQ=5   -> REJECTED (too old, outside window)
```

Program 14: Packet Analyzer for AH and ESP Headers

Aim: Develop a packet analyzer that identifies AH and ESP headers in captured IP packets.

Requirements:

- Inspect the IP protocol field of captured packets.
- Classify each packet as AH (51), ESP (50), or other.
- Summarize the counts of each protocol type observed.

Expected Learning Outcome: Understand how AH/ESP traffic can be identified from captured packets.

```
# Program 14: Packet Analyzer to Identify AH and ESP Headers
captured_packets = [
    {"id": 1, "ip_proto": 51, "src": "10.0.0.1", "dst": "10.0.0.2"},   # 51 = AH
    {"id": 2, "ip_proto": 50, "src": "10.0.0.3", "dst": "10.0.0.4"},   # 50 = ESP
    {"id": 3, "ip_proto": 6,  "src": "10.0.0.5", "dst": "10.0.0.6"},   # TCP
    {"id": 4, "ip_proto": 51, "src": "10.0.0.7", "dst": "10.0.0.8"},
    {"id": 5, "ip_proto": 17, "src": "10.0.0.9", "dst": "10.0.0.10"},  # UDP
]

PROTO_NAMES = {51: "AH", 50: "ESP", 6: "TCP", 17: "UDP"}

def analyze(packet):
    name = PROTO_NAMES.get(packet["ip_proto"], "OTHER")
    return name

print("=== IPSec Packet Analyzer ===")
ah_count = esp_count = other_count = 0
for pkt in captured_packets:
    proto = analyze(pkt)
    print(f"Packet #{pkt['id']}: {pkt['src']} -> {pkt['dst']}  IP-Proto={pkt['ip_proto']}
        ({proto})")
    if proto == "AH": ah_count += 1
    elif proto == "ESP": esp_count += 1
    else: other_count += 1

print(f"\nSummary: AH packets={ah_count}, ESP packets={esp_count}, Other={other_count}")
```

```
=== IPSec Packet Analyzer ===
Packet #1: 10.0.0.1 -> 10.0.0.2  IP-Proto=51 (AH)
Packet #2: 10.0.0.3 -> 10.0.0.4  IP-Proto=50 (ESP)
Packet #3: 10.0.0.5 -> 10.0.0.6  IP-Proto=6 (TCP)
Packet #4: 10.0.0.7 -> 10.0.0.8  IP-Proto=51 (AH)
Packet #5: 10.0.0.9 -> 10.0.0.10  IP-Proto=17 (UDP)

Summary: AH packets=2, ESP packets=1, Other=2
```

Program 15: AH vs ESP Performance Comparison

Aim: Compare the performance of AH and ESP by measuring processing time and packet overhead.

Requirements:

- Measure total and average per-packet processing time for AH (HMAC) and ESP (AES encryption).
- Measure the per-packet byte overhead added by each protocol.

- Summarize the trade-offs between the two protocols.

Expected Learning Outcome: Understand the performance/security trade-off between AH and ESP.

```python
# Program 15: Compare Performance of AH and ESP (processing time & overhead)
import os, time, hmac, hashlib
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.primitives import padding as sympad

N = 2000
payload = os.urandom(512)
ah_key = os.urandom(32)
esp_key = os.urandom(16)

start = time.perf_counter()
for _ in range(N):
    icv = hmac.new(ah_key, payload, hashlib.sha256).digest()
ah_time = time.perf_counter() - start
ah_overhead = len(icv)  # bytes added

start = time.perf_counter()
for _ in range(N):
    iv = os.urandom(16)
    padder = sympad.PKCS7(128).padder()
    padded = padder.update(payload) + padder.finalize()
    cipher = Cipher(algorithms.AES(esp_key), modes.CBC(iv))
    ct = cipher.encryptor().update(padded) + cipher.encryptor().finalize()
esp_time = time.perf_counter() - start
esp_overhead = len(iv) + (len(ct) - len(payload))

print(f"=== AH vs ESP Performance Comparison ({N} packets of {len(payload)} bytes) ===")
print(f"{'Metric':<25}{'AH':<20}{'ESP'}")
print(f"{'Total time (s)':<25}{ah_time:<20.4f}{esp_time:.4f}")
print(f"{'Avg time/packet (ms)':<25}{ah_time/N*1000:<20.4f}{esp_time/N*1000:.4f}")
print(f"{'Overhead/packet (bytes)':<25}{ah_overhead:<20}{esp_overhead}")
print("\nObservation: AH (hashing only) is faster and lighter than ESP (encryption + padding
    + IV),")
print("but AH does not provide confidentiality while ESP does.")
```

```
=== AH vs ESP Performance Comparison (2000 packets of 512 bytes) ===
Metric                   AH                  ESP
Total time (s)           0.0097              0.0307
Avg time/packet (ms)     0.0049              0.0154
Overhead/packet (bytes)  32                  32

Observation: AH (hashing only) is faster and lighter than ESP (encryption + padding + IV),
but AH does not provide confidentiality while ESP does.
```

Email Security

Program 16: Email Header Analysis

Aim: Develop a program to analyze an email header and identify sender, receiver, routing path, and timestamps.

Requirements:

- Parse a raw email header using Python's email library.
- Extract From, To, Subject and Date fields.
- Reconstruct the routing path from the Received header chain.

Expected Learning Outcome: Understand how to read an email header to trace its origin and path.

```python
# Program 16: Analyze an Email Header - Sender, Receiver, Routing Path, Timestamps
from email import message_from_string

raw_header = """From: alice@example.com
To: bob@company.com
Subject: Project Update
Date: Mon, 03 Aug 2026 14:22:10 +0530
Received: from mx1.company.com (mx1.company.com [203.0.113.5]) by mail.company.com; Mon, 03
    Aug 2026 14:22:15 +0530
Received: from smtp.example.com (smtp.example.com [198.51.100.20]) by mx1.company.com; Mon,
    03 Aug 2026 14:22:12 +0530
Received: from client.example.com ([192.0.2.44]) by smtp.example.com; Mon, 03 Aug 2026
    14:22:09 +0530
"""

msg = message_from_string(raw_header)

print("=== Email Header Analysis ===")
print("Sender      :", msg.get("From"))
print("Receiver    :", msg.get("To"))
print("Subject     :", msg.get("Subject"))
print("Date        :", msg.get("Date"))

print("\nRouting Path (Received headers, origin -> destination):")
received_list = msg.get_all("Received")
for i, hop in enumerate(reversed(received_list), start=1):
    print(f"  Hop {i}: {hop.split(' by ')[0].replace('from ', '')}")

print(f"\nTotal hops traversed: {len(received_list)}")
```

```
=== Email Header Analysis ===
Sender      : alice@example.com
Receiver    : bob@company.com
Subject     : Project Update
Date        : Mon, 03 Aug 2026 14:22:10 +0530

Routing Path (Received headers, origin -> destination):
  Hop 1: client.example.com ([192.0.2.44])
  Hop 2: smtp.example.com (smtp.example.com [198.51.100.20])
  Hop 3: mx1.company.com (mx1.company.com [203.0.113.5])

Total hops traversed: 3
```

Program 17: Authentication Field Parser

Aim: Implement a parser that extracts authentication-related fields from an email header.

Requirements:

- Extract Authentication-Results, DKIM-Signature and Received-SPF fields.

- Parse individual SPF, DKIM and DMARC verdicts from Authentication-Results.

Expected Learning Outcome: Understand how authentication metadata is embedded in email headers.

```
# Program 17: Parser Extracting Authentication-Related Fields from Email Header
import re

raw_header = """From: notifications@bank-secure.com
To: user@example.com
Authentication-Results: mx.example.com; spf=pass smtp.mailfrom=bank-secure.com; dkim=pass
    header.d=bank-secure.com; dmarc=pass
DKIM-Signature: v=1; a=rsa-sha256; d=bank-secure.com; s=selector1; h=from:to:subject;
    bh=base64hash==; b=signaturevalue==
Received-SPF: pass (google.com: domain of bank-secure.com designates 203.0.113.9 as
    permitted sender)
"""

def extract_field(header_text, field_name):
    match = re.search(rf"^{field_name}:\s*(.*)$", header_text, re.MULTILINE)
    return match.group(1) if match else None

print("=== Authentication Field Extraction ===")
auth_results = extract_field(raw_header, "Authentication-Results")
dkim_sig = extract_field(raw_header, "DKIM-Signature")
spf = extract_field(raw_header, "Received-SPF")

print("Authentication-Results:", auth_results)
print("DKIM-Signature        :", dkim_sig)
print("Received-SPF          :", spf)

print("\nParsed verdicts:")
for check in ["spf", "dkim", "dmarc"]:
    m = re.search(rf"{check}=(\w+)", auth_results)
    print(f"  {check.upper():5}: {m.group(1) if m else 'not present'}")
```

```
=== Authentication Field Extraction ===
Authentication-Results: mx.example.com; spf=pass smtp.mailfrom=bank-secure.com; dkim=pass
    header.d=bank-secure.com; dmarc=pass
DKIM-Signature        : v=1; a=rsa-sha256; d=bank-secure.com; s=selector1;
    h=from:to:subject; bh=base64hash==; b=signaturevalue==
Received-SPF          : pass (google.com: domain of bank-secure.com designates 203.0.113.9
    as permitted sender)

Parsed verdicts:
  SPF  : pass
  DKIM : pass
  DMARC: pass
```

Program 18: Spoofed Email Address Detection

Aim: Create a tool to detect spoofed email addresses by analyzing email headers.

Requirements:

- Compare the From domain against the Return-Path domain.
- Check SPF and DKIM verification results.

- Flag mismatches between the sending server and claimed domain.

Expected Learning Outcome: Understand practical indicators used to detect email spoofing.

```python
# Program 18: Detect Spoofed Email Addresses by Analyzing Email Headers
emails = [
    {"from": "support@paypal.com", "return_path": "support@paypal.com", "spf": "pass",
        "dkim": "pass", "received_domain": "paypal.com"},
    {"from": "support@paypal.com", "return_path": "attacker@evil-mail.ru", "spf": "fail",
        "dkim": "fail", "received_domain": "evil-mail.ru"},
    {"from": "billing@amazon.com", "return_path": "billing@amaz0n-billing.com", "spf":
        "fail", "dkim": "none", "received_domain": "amaz0n-billing.com"},
    {"from": "hr@company.com", "return_path": "hr@company.com", "spf": "pass", "dkim":
        "pass", "received_domain": "company.com"},
]

def from_domain(addr):
    return addr.split("@")[1]

def is_spoofed(mail):
    reasons = []
    if from_domain(mail["from"]) != from_domain(mail["return_path"]):
        reasons.append("From domain != Return-Path domain")
    if mail["spf"] != "pass":
        reasons.append("SPF check failed")
    if mail["dkim"] != "pass":
        reasons.append("DKIM check failed")
    if from_domain(mail["from"]) != mail["received_domain"]:
        reasons.append("Sending server domain mismatch")
    return reasons

print("=== Spoofed Email Detection ===")
for mail in emails:
    reasons = is_spoofed(mail)
    verdict = "SPOOFED / SUSPICIOUS" if reasons else "LEGITIMATE"
    print(f"\nFrom: {mail['from']}  |  Verdict: {verdict}")
    for r in reasons:
        print(f"   - {r}")
```

```
=== Spoofed Email Detection ===

From: support@paypal.com  |  Verdict: LEGITIMATE

From: support@paypal.com  |  Verdict: SPOOFED / SUSPICIOUS
   - From domain != Return-Path domain
   - SPF check failed
   - DKIM check failed
   - Sending server domain mismatch

From: billing@amazon.com  |  Verdict: SPOOFED / SUSPICIOUS
   - From domain != Return-Path domain
   - SPF check failed
   - DKIM check failed
   - Sending server domain mismatch

From: hr@company.com  |  Verdict: LEGITIMATE
```

Program 19: Working Principle of PGP

Aim: Develop a simulation demonstrating the working principle of Pretty Good Privacy (PGP).

Requirements:

- Show message digesting and signing with the sender's private key.
- Show session-key generation and symmetric encryption of the message.
- Show session-key encryption with the receiver's public key and the full receive-side reversal.

Expected Learning Outcome: Understand PGP's hybrid use of symmetric and asymmetric cryptography.

```python
# Program 19: Simulation Demonstrating the Working Principle of PGP
import hashlib, os

print("=== PGP Working Principle Simulation ===\n")

message = "Meeting rescheduled to 10 AM tomorrow."
print("Step 1: Sender composes message:", message)

digest = hashlib.sha256(message.encode()).hexdigest()
print(f"\nStep 2: Sender computes message digest (SHA-256): {digest[:32]}...")
print("Step 3: Digest signed with sender's PRIVATE key -> digital signature")

session_key = os.urandom(16)
print(f"\nStep 4: Random one-time session key generated: {session_key.hex()}")
print("Step 5: Message + signature encrypted using session key (symmetric, fast)")
print("Step 6: Session key itself encrypted using receiver's PUBLIC key (asymmetric)")

print("\nStep 7: Package sent = [Encrypted session key][Encrypted(message+signature)]")

print("\n--- At receiver ---")
print("Step 8: Receiver decrypts session key using their PRIVATE key")
print("Step 9: Session key used to decrypt message + signature")
print("Step 10: Signature verified using sender's PUBLIC key -> authenticates sender")
print("Step 11: Digest recomputed and compared to verify integrity")
print("\nPGP achieves confidentiality (session key + public key encryption),")
print("authentication and integrity (digital signature) in one scheme.")
```

```
=== PGP Working Principle Simulation ===

Step 1: Sender composes message: Meeting rescheduled to 10 AM tomorrow.

Step 2: Sender computes message digest (SHA-256): a7eb92c94a4a57a72c4a34a2bcf16b11...
Step 3: Digest signed with sender's PRIVATE key -> digital signature

Step 4: Random one-time session key generated: 5bfb25963260440ed0a9ae602f79627d
Step 5: Message + signature encrypted using session key (symmetric, fast)
Step 6: Session key itself encrypted using receiver's PUBLIC key (asymmetric)

Step 7: Package sent = [Encrypted session key][Encrypted(message+signature)]

--- At receiver ---
Step 8: Receiver decrypts session key using their PRIVATE key
Step 9: Session key used to decrypt message + signature
Step 10: Signature verified using sender's PUBLIC key -> authenticates sender
Step 11: Digest recomputed and compared to verify integrity

PGP achieves confidentiality (session key + public key encryption),
authentication and integrity (digital signature) in one scheme.
```

Program 20: PGP-based Email Encryption and Decryption

Aim: Implement PGP-based email encryption and decryption using public and private keys.

Requirements:

- Generate an RSA key pair for the receiver.
- Encrypt a random AES session key with RSA-OAEP and use it to AES-encrypt the message.
- Decrypt the session key and message at the receiver and confirm correctness.

Expected Learning Outcome: Understand hybrid PGP encryption/decryption using RSA and AES.

```python
# Program 20: PGP-based Email Encryption and Decryption using Public/Private Keys
import os
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.primitives import padding as sympad

# Receiver generates RSA keypair
receiver_key = rsa.generate_private_key(public_exponent=65537, key_size=2048)
receiver_pub = receiver_key.public_key()

def pgp_encrypt(message, pub_key):
    session_key = os.urandom(32)
    iv = os.urandom(16)
    padder = sympad.PKCS7(128).padder()
    padded = padder.update(message.encode()) + padder.finalize()
    cipher = Cipher(algorithms.AES(session_key), modes.CBC(iv))
    ciphertext = cipher.encryptor().update(padded) + cipher.encryptor().finalize()
    enc_session_key = pub_key.encrypt(session_key, padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()), algorithm=hashes.SHA256(), label=None))
    return enc_session_key, iv, ciphertext

def pgp_decrypt(enc_session_key, iv, ciphertext, priv_key):
    session_key = priv_key.decrypt(enc_session_key, padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()), algorithm=hashes.SHA256(), label=None))
    cipher = Cipher(algorithms.AES(session_key), modes.CBC(iv))
    padded = cipher.decryptor().update(ciphertext) + cipher.decryptor().finalize()
    unpadder = sympad.PKCS7(128).unpadder()
    return (unpadder.update(padded) + unpadder.finalize()).decode()

email_body = "Attached is the signed contract. Please countersign and return by Friday."
print("=== PGP Email Encryption/Decryption ===")
print("Original email:", email_body)

enc_session_key, iv, ciphertext = pgp_encrypt(email_body, receiver_pub)
print(f"\nEncrypted session key (RSA-OAEP): {enc_session_key.hex()[:32]}...")
print(f"Encrypted email body (AES-256-CBC): {ciphertext.hex()[:32]}...")

decrypted = pgp_decrypt(enc_session_key, iv, ciphertext, receiver_key)
print("\nDecrypted email at receiver:", decrypted)
print("Match with original:", decrypted == email_body)
```

```
=== PGP Email Encryption/Decryption ===
Original email: Attached is the signed contract. Please countersign and return by Friday.

Encrypted session key (RSA-OAEP): 14c448cb10c9dbdab46f6b0392521bcd...
Encrypted email body (AES-256-CBC): 6d1bbf92bd97e02f0385a933f7956d95...

Decrypted email at receiver: Attached is the signed contract. Please countersign and return
    by Friday.
Match with original: True
```

Program 21: PGP Digital Signature Generation and Verification

Aim: Write a program to generate and verify digital signatures using the PGP framework.

Requirements:

- Sign a message using RSA-PSS with the sender's private key.
- Verify the signature using the sender's public key.
- Show that verification fails for a tampered message.

Expected Learning Outcome: Understand how digital signatures provide authentication and integrity.

```python
# Program 21: Generate and Verify Digital Signatures using PGP Framework
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes
from cryptography.exceptions import InvalidSignature

sender_key = rsa.generate_private_key(public_exponent=65537, key_size=2048)
sender_pub = sender_key.public_key()

def sign_message(message, priv_key):
    return priv_key.sign(message.encode(), padding.PSS(
        mgf=padding.MGF1(hashes.SHA256()), salt_length=padding.PSS.MAX_LENGTH),
            hashes.SHA256())

def verify_signature(message, signature, pub_key):
    try:
        pub_key.verify(signature, message.encode(), padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()), salt_length=padding.PSS.MAX_LENGTH),
                hashes.SHA256())
        return True
    except InvalidSignature:
        return False

message = "I approve the release of build v2.4.1"
print("=== PGP Digital Signature Generation & Verification ===")
print("Message:", message)

signature = sign_message(message, sender_key)
print(f"\nGenerated signature (RSA-PSS, SHA-256): {signature.hex()[:32]}...")

print("\nVerifying with correct message...")
print("Signature valid:", verify_signature(message, signature, sender_pub))

print("\nVerifying with tampered message...")
print("Signature valid:", verify_signature("I approve the release of build v9.9.9",
    signature, sender_pub))
```

```
=== PGP Digital Signature Generation & Verification ===
Message: I approve the release of build v2.4.1

Generated signature (RSA-PSS, SHA-256): 31582676d9788c769896b99c973291b7...

Verifying with correct message...
Signature valid: True

Verifying with tampered message...
Signature valid: False
```

Program 22: Secure Email Transmission using the PGP Model

Aim: Develop a secure email transmission system using the PGP model.

Requirements:

- Sign the message, then encrypt message+signature with a session key.
- Encrypt the session key with the receiver's RSA public key.
- Decrypt and verify the signature at the receiver end-to-end.

Expected Learning Outcome: Understand a complete PGP-style secure send/receive pipeline.

```python
# Program 22: Secure Email Transmission System using the PGP Model
import os
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.primitives import padding as sympad
from cryptography.exceptions import InvalidSignature

sender_key = rsa.generate_private_key(public_exponent=65537, key_size=2048)
receiver_key = rsa.generate_private_key(public_exponent=65537, key_size=2048)

def pgp_send(message, sender_priv, receiver_pub):
    signature = sender_priv.sign(message.encode(), padding.PSS(
        mgf=padding.MGF1(hashes.SHA256()), salt_length=padding.PSS.MAX_LENGTH),
            hashes.SHA256())
    payload = message.encode() + b"||SIG||" + signature
    session_key, iv = os.urandom(32), os.urandom(16)
    padder = sympad.PKCS7(128).padder()
    padded = padder.update(payload) + padder.finalize()
    ct = Cipher(algorithms.AES(session_key), modes.CBC(iv)).encryptor()
    ciphertext = ct.update(padded) + ct.finalize()
    enc_key = receiver_pub.encrypt(session_key, padding.OAEP(
        mgf=padding.MGF1(hashes.SHA256()), algorithm=hashes.SHA256(), label=None))
    return enc_key, iv, ciphertext

def pgp_receive(enc_key, iv, ciphertext, receiver_priv, sender_pub):
    session_key = receiver_priv.decrypt(enc_key, padding.OAEP(
        mgf=padding.MGF1(hashes.SHA256()), algorithm=hashes.SHA256(), label=None))
    dec = Cipher(algorithms.AES(session_key), modes.CBC(iv)).decryptor()
    padded = dec.update(ciphertext) + dec.finalize()
    unpadder = sympad.PKCS7(128).unpadder()
    payload = unpadder.update(padded) + unpadder.finalize()
    message, signature = payload.split(b"||SIG||")
    try:
        sender_pub.verify(signature, message, padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()), salt_length=padding.PSS.MAX_LENGTH),
                hashes.SHA256())
        valid = True
    except InvalidSignature:
        valid = False
    return message.decode(), valid

email = "Board meeting minutes attached, please review before Friday."
print("=== End-to-End Secure Email Transmission (PGP model) ===")
print("Sending:", email)
enc_key, iv, ciphertext = pgp_send(email, sender_key, receiver_key.public_key())
print(f"Transmitted encrypted payload: {ciphertext.hex()[:32]}...")

recovered, sig_valid = pgp_receive(enc_key, iv, ciphertext, receiver_key,
    sender_key.public_key())
print("\nReceived & decrypted:", recovered)
print("Signature verified (sender authenticated):", sig_valid)
```

```
=== End-to-End Secure Email Transmission (PGP model) ===
Sending: Board meeting minutes attached, please review before Friday.
Transmitted encrypted payload: 8fd67b9c605efa24e0f66acfac2db561...

Received & decrypted: Board meeting minutes attached, please review before Friday.
Signature verified (sender authenticated): True
```

Program 23: S/MIME Email Encryption and Digital Signature Verification

Aim: Implement S/MIME-based email encryption and digital signature verification.

Requirements:

- Generate an X.509 certificate for the sender.
- Sign a message with the certificate's private key.
- Encrypt the message and verify the signature using the certificate's public key.

Expected Learning Outcome: Understand how S/MIME uses X.509 certificates for signing and encryption.

```python
# Program 23: S/MIME-based Email Encryption and Digital Signature Verification
import os, datetime
from cryptography import x509
from cryptography.x509.oid import NameOID
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.primitives import padding as sympad

# Generate a self-signed X.509 certificate for the sender (S/MIME uses CA-issued certs)
priv_key = rsa.generate_private_key(public_exponent=65537, key_size=2048)
subject = issuer = x509.Name([x509.NameAttribute(NameOID.COMMON_NAME, "alice@corp.com")])
cert = (x509.CertificateBuilder()
        .subject_name(subject).issuer_name(issuer).public_key(priv_key.public_key())
        .serial_number(x509.random_serial_number())
        .not_valid_before(datetime.datetime.utcnow())
        .not_valid_after(datetime.datetime.utcnow() + datetime.timedelta(days=365))
        .sign(priv_key, hashes.SHA256()))

print("=== S/MIME Certificate ===")
print("Subject:", cert.subject.rfc4514_string())
print("Valid until:", cert.not_valid_after)

message = "Invoice #4521 attached, payable within 30 days."
signature = priv_key.sign(message.encode(), padding.PKCS1v15(), hashes.SHA256())
print("\nMessage signed with private key corresponding to certificate.")

session_key, iv = os.urandom(32), os.urandom(16)
padder = sympad.PKCS7(128).padder()
padded = padder.update(message.encode()) + padder.finalize()
enc = Cipher(algorithms.AES(session_key), modes.CBC(iv)).encryptor()
ciphertext = enc.update(padded) + enc.finalize()
print(f"Encrypted message (S/MIME envelope): {ciphertext.hex()[:32]}...")

print("\n=== Receiver Verification ===")
cert.public_key().verify(signature, message.encode(), padding.PKCS1v15(), hashes.SHA256())
print("Certificate chain trusted, digital signature VALID -> sender authenticated & message
    unmodified.")
```

```
=== S/MIME Certificate ===
Subject: CN=alice@corp.com
Valid until: 2027-08-04 03:37:00

Message signed with private key corresponding to certificate.
Encrypted message (S/MIME envelope): e7f5e5d8979f848b410ae02df62a0876...

=== Receiver Verification ===
Certificate chain trusted, digital signature VALID -> sender authenticated & message
    unmodified.
```

Program 24: Comparison of PGP and S/MIME

Aim: Create a simulation to compare PGP and S/MIME based on encryption, authentication, and certificate management.

Requirements:

- Compare trust models: Web of Trust versus hierarchical CA.
- Compare key distribution, certificate management and typical usage.

Expected Learning Outcome: Understand the practical differences between PGP and S/MIME.

```
# Program 24: Compare PGP and S/MIME (Encryption, Authentication, Certificate Management)
comparison = [
    ("Trust model",        "Web of Trust (users sign each other's keys)", "Hierarchical CA-
        issued X.509 certificates"),
    ("Key distribution",   "Public key servers / manual exchange",        "Certificate
        Authority (CA)"),
    ("Encryption",         "RSA/ElGamal + session key (AES/CAST5)",       "RSA/ECC + session
        key (AES)"),
    ("Signature",          "RSA/DSA digital signature",                   "RSA/ECDSA with
        X.509 certificate"),
    ("Certificate mgmt",   "No formal certificate, self-generated keys",  "Formal
        certificates, revocation via CRL/OCSP"),
    ("Common usage",       "Personal/independent email, open-source",     "Enterprise &
        government email systems"),
    ("Interoperability",   "Requires PGP-compatible client (GPG, etc.)",  "Built into most
        enterprise mail clients"),
]

print("=== PGP vs S/MIME Comparison ===\n")
print(f"{'Aspect':<20}{'PGP':<48}{'S/MIME'}")
print("-" * 110)
for aspect, pgp, smime in comparison:
    print(f"{aspect:<20}{pgp:<48}{smime}")
```

```
=== PGP vs S/MIME Comparison ===

Aspect              PGP                                              S/MIME
--------------------------------------------------------------------------------------------------------------
Trust model         Web of Trust (users sign each other's keys)      Hierarchical CA-issued
    X.509 certificates
Key distribution    Public key servers / manual exchange             Certificate Authority
    (CA)
Encryption          RSA/ElGamal + session key (AES/CAST5)            RSA/ECC + session key
    (AES)
Signature           RSA/DSA digital signature                        RSA/ECDSA with X.509
    certificate
Certificate mgmt    No formal certificate, self-generated keys       Formal certificates,
    revocation via CRL/OCSP
Common usage        Personal/independent email, open-source          Enterprise & government
    email systems
Interoperability    Requires PGP-compatible client (GPG, etc.)       Built into most
    enterprise mail clients
```

Program 25: Certificate Validation Module for S/MIME

Aim: Develop a certificate validation module for S/MIME email communication.

Requirements:

- Issue certificates signed by a simulated Certificate Authority.

- Validate expiry and issuer trust for each certificate.
- Demonstrate rejection of an expired certificate.

Expected Learning Outcome: Understand how certificate validation underpins trust in S/MIME.

```python
# Program 25: Certificate Validation Module for S/MIME Email Communication
import datetime
from cryptography import x509
from cryptography.x509.oid import NameOID
from cryptography.hazmat.primitives.asymmetric import rsa
from cryptography.hazmat.primitives import hashes

def make_cert(cn, not_before_days, not_after_days, issuer_key=None, issuer_name=None):
    key = rsa.generate_private_key(public_exponent=65537, key_size=2048)
    name = x509.Name([x509.NameAttribute(NameOID.COMMON_NAME, cn)])
    signer_key = issuer_key or key
    signer_name = issuer_name or name
    cert = (x509.CertificateBuilder()
            .subject_name(name).issuer_name(signer_name).public_key(key.public_key())
            .serial_number(x509.random_serial_number())
            .not_valid_before(datetime.datetime.utcnow() +
                datetime.timedelta(days=not_before_days))
            .not_valid_after(datetime.datetime.utcnow() +
                datetime.timedelta(days=not_after_days))
            .sign(signer_key, hashes.SHA256()))
    return key, cert

ca_key, ca_cert = make_cert("Corp Root CA", -1, 3650)
user_key, user_cert = make_cert("bob@corp.com", -1, 365, issuer_key=ca_key,
    issuer_name=ca_cert.subject)
_, expired_cert = make_cert("carol@corp.com", -60, -30, issuer_key=ca_key,
    issuer_name=ca_cert.subject)  # already expired

def validate_certificate(cert, trusted_issuer_name):
    now = datetime.datetime.utcnow()
    checks = {
        "Not expired": cert.not_valid_before <= now <= cert.not_valid_after,
        "Issued by trusted CA": cert.issuer == trusted_issuer_name,
    }
    return checks

print("=== S/MIME Certificate Validation ===\n")
for label, cert in [("Bob's certificate", user_cert), ("Carol's certificate (expired)",
    expired_cert)]:
    print(f"{label}: subject={cert.subject.rfc4514_string()}")
    checks = validate_certificate(cert, ca_cert.subject)
    for check, passed in checks.items():
        print(f"   {check}: {'PASS' if passed else 'FAIL'}")
    print("   Overall:", "VALID - trusted for S/MIME" if all(checks.values()) else "INVALID
        - rejected")
    print()
```

```
=== S/MIME Certificate Validation ===

Bob's certificate: subject=CN=bob@corp.com
   Not expired: PASS
   Issued by trusted CA: PASS
   Overall: VALID - trusted for S/MIME

Carol's certificate (expired): subject=CN=carol@corp.com
   Not expired: FAIL
   Issued by trusted CA: PASS
   Overall: INVALID - rejected
```

Email Spam Detection

Program 26: Keyword-Based Spam Filtering

Aim: Implement a spam detection system using keyword-based filtering techniques.

Requirements:

- Maintain a list of common spam trigger words/phrases.
- Score each email by counting matched keywords.
- Classify the email as SPAM or HAM based on a threshold.

Expected Learning Outcome: Understand the strengths and limitations of simple keyword-based filtering.

```python
# Program 26: Spam Detection using Keyword-Based Filtering
SPAM_KEYWORDS = ["free", "winner", "congratulations", "click here", "urgent",
                 "act now", "lottery", "viagra", "limited offer", "claim your prize"]

def keyword_score(text):
    text_lower = text.lower()
    matched = [kw for kw in SPAM_KEYWORDS if kw in text_lower]
    return matched

def classify(text, threshold=1):
    matched = keyword_score(text)
    return ("SPAM" if len(matched) >= threshold else "HAM"), matched

emails = [
    "Congratulations! You are the winner of our lottery. Click here to claim your prize
        now!",
    "Hi team, please find the attached report for this week's sprint review.",
    "URGENT: act now for a limited offer, free trial ends today!",
    "Reminder: your dentist appointment is scheduled for tomorrow at 10 AM.",
]

print("=== Keyword-Based Spam Filter ===")
for email in emails:
    verdict, matched = classify(email)
    print(f"\nEmail: \"{email}\"")
    print(f"  Matched keywords: {matched if matched else 'none'}")
    print(f"  Verdict: {verdict}")
```

```
=== Keyword-Based Spam Filter ===

Email: "Congratulations! You are the winner of our lottery. Click here to claim your prize
    now!"
  Matched keywords: ['winner', 'congratulations', 'click here', 'lottery', 'claim your
      prize']
  Verdict: SPAM

Email: "Hi team, please find the attached report for this week's sprint review."
  Matched keywords: none
  Verdict: HAM

Email: "URGENT: act now for a limited offer, free trial ends today!"
  Matched keywords: ['free', 'urgent', 'act now', 'limited offer']
  Verdict: SPAM

Email: "Reminder: your dentist appointment is scheduled for tomorrow at 10 AM."
  Matched keywords: none
  Verdict: HAM
```

Program 27: Naive Bayes Spam Classifier

Aim: Develop a machine learning-based spam classifier using the Naive Bayes algorithm.

Requirements:

- Vectorize a labeled set of spam/ham emails using bag-of-words.

- Train a Multinomial Naive Bayes classifier and evaluate accuracy.
- Classify a new, unseen email.

Expected Learning Outcome: Understand how Naive Bayes is applied to text-based spam classification.

```python
# Program 27: Machine Learning-based Spam Classifier using Naive Bayes
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

emails = [
    "Congratulations you have won a free lottery prize claim now",
    "Limited time offer click here to win cash urgent",
    "Free viagra pills discount buy now act fast",
    "You are selected winner claim your reward today",
    "Get rich quick work from home no experience needed",
    "Meeting rescheduled to 3 PM tomorrow please confirm",
    "Please find attached the quarterly financial report",
    "Can we discuss the project timeline this week",
    "Your package has been shipped and will arrive Monday",
    "Reminder your dentist appointment is tomorrow morning",
]
labels = [1, 1, 1, 1, 1, 0, 0, 0, 0, 0]  # 1 = spam, 0 = ham

X_train, X_test, y_train, y_test = train_test_split(emails, labels, test_size=0.3,
    random_state=42)

vectorizer = CountVectorizer()
X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)

model = MultinomialNB()
model.fit(X_train_vec, y_train)
predictions = model.predict(X_test_vec)

print("=== Naive Bayes Spam Classifier ===")
for text, true_label, pred in zip(X_test, y_test, predictions):
    print(f"'{text[:50]}...' -> Actual={'SPAM' if true_label else 'HAM'}, Predicted={'SPAM'
        if pred else 'HAM'}")

print(f"\nAccuracy: {accuracy_score(y_test, predictions) * 100:.2f}%")

new_email = ["Free money click here to claim your prize"]
result = model.predict(vectorizer.transform(new_email))
print(f"\nNew email: '{new_email[0]}' -> {'SPAM' if result[0] else 'HAM'}")
```

```
=== Naive Bayes Spam Classifier ===
'Your package has been shipped and will arrive Mond...' -> Actual=HAM, Predicted=SPAM
'Limited time offer click here to win cash urgent...' -> Actual=SPAM, Predicted=SPAM
'Meeting rescheduled to 3 PM tomorrow please confir...' -> Actual=HAM, Predicted=HAM

Accuracy: 66.67%

New email: 'Free money click here to claim your prize' -> SPAM
```

Program 28: SVM Spam Classifier vs Naive Bayes

Aim: Create a spam detection application using Support Vector Machine (SVM) and compare its accuracy with Naive Bayes.

Requirements:

- Train both an SVM (linear kernel) and a Naive Bayes classifier on the same data.
- Compare predictions and accuracy of both classifiers on a held-out test set.

Expected Learning Outcome: Understand comparative performance of SVM and Naive Bayes for spam detection.

```python
# Program 28: Spam Detection using SVM, Compared with Naive Bayes
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

emails = [
    "Congratulations you have won a free lottery prize claim now",
    "Limited time offer click here to win cash urgent",
    "Free viagra pills discount buy now act fast",
    "You are selected winner claim your reward today",
    "Get rich quick work from home no experience needed",
    "Exclusive deal free gift card only for you today",
    "Meeting rescheduled to 3 PM tomorrow please confirm",
    "Please find attached the quarterly financial report",
    "Can we discuss the project timeline this week",
    "Your package has been shipped and will arrive Monday",
    "Reminder your dentist appointment is tomorrow morning",
    "Team lunch is planned for Friday afternoon",
]
labels = [1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0]

X_train, X_test, y_train, y_test = train_test_split(emails, labels, test_size=0.3,
    random_state=1)
vectorizer = CountVectorizer()
X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)

nb_model = MultinomialNB().fit(X_train_vec, y_train)
svm_model = SVC(kernel="linear").fit(X_train_vec, y_train)

nb_preds = nb_model.predict(X_test_vec)
svm_preds = svm_model.predict(X_test_vec)

print("=== SVM vs Naive Bayes Spam Classification ===")
for text, true_label, nbp, svp in zip(X_test, y_test, nb_preds, svm_preds):
    print(f"'{text[:45]}...' Actual={'SPAM' if true_label else 'HAM':4} NB={'SPAM' if nbp
        else 'HAM':4} SVM={'SPAM' if svp else 'HAM'}")

print(f"\nNaive Bayes Accuracy: {accuracy_score(y_test, nb_preds) * 100:.2f}%")
print(f"SVM Accuracy        : {accuracy_score(y_test, svm_preds) * 100:.2f}%")
```

```
=== SVM vs Naive Bayes Spam Classification ===
'Free viagra pills discount buy now act fast...' Actual=SPAM NB=SPAM SVM=SPAM
'You are selected winner claim your reward tod...' Actual=SPAM NB=SPAM SVM=SPAM
'Get rich quick work from home no experience n...' Actual=SPAM NB=HAM  SVM=HAM
'Reminder your dentist appointment is tomorrow...' Actual=HAM  NB=HAM  SVM=HAM

Naive Bayes Accuracy: 75.00%
SVM Accuracy        : 75.00%
```

Program 29: Phishing Email Detection

Aim: Design a phishing email detection system by analyzing suspicious URLs, attachments, and email headers.

Requirements:

- Flag URLs using IP addresses, excessive subdomains, '@' obfuscation, or lookalike domains.
- Flag suspicious attachment file types.
- Combine URL, attachment and SPF signals into a phishing score and verdict.

Expected Learning Outcome: Understand multi-signal analysis used in phishing detection.

```python
# Program 29: Phishing Email Detection using URLs, Attachments and Headers
import re

SUSPICIOUS_EXTENSIONS = [".exe", ".scr", ".js", ".vbs", ".bat"]

def analyze_url(url):
    flags = []
    if re.match(r"https?://\d+\.\d+\.\d+\.\d+", url):
        flags.append("IP address used instead of domain")
    if url.count(".") > 4:
        flags.append("Excessive subdomains")
    if "@" in url:
        flags.append("'@' symbol used to obscure real destination")
    if any(bad in url for bad in ["paypa1", "amaz0n", "secure-login", "verify-account"]):
        flags.append("Lookalike / suspicious keyword domain")
    return flags

def analyze_attachment(filename):
    return any(filename.endswith(ext) for ext in SUSPICIOUS_EXTENSIONS)

def analyze_email(email):
    score = 0
    reasons = []
    for url in email.get("urls", []):
        f = analyze_url(url)
        if f:
            score += len(f)
            reasons.extend(f"URL '{url}': {x}" for x in f)
    for att in email.get("attachments", []):
        if analyze_attachment(att):
            score += 2
            reasons.append(f"Suspicious attachment type: {att}")
    if email.get("spf") == "fail":
        score += 2
        reasons.append("SPF authentication failed")
    return score, reasons

emails = [
    {"from": "security@paypa1-verify.com", "urls": ["http://192.168.5.9/login",
        "http://secure-login.paypa1.com/verify"],
     "attachments": ["invoice.exe"], "spf": "fail"},
    {"from": "hr@company.com", "urls": ["https://intranet.company.com/policies"],
     "attachments": ["handbook.pdf"], "spf": "pass"},
]

print("=== Phishing Email Detection ===")
for email in emails:
    score, reasons = analyze_email(email)
    verdict = "PHISHING" if score >= 3 else ("SUSPICIOUS" if score > 0 else "LEGITIMATE")
    print(f"\nFrom: {email['from']}  |  Score: {score}  |  Verdict: {verdict}")
    for r in reasons:
        print("   -", r)
```

```
=== Phishing Email Detection ===

From: security@paypa1-verify.com  |  Score: 6  |  Verdict: PHISHING
   - URL 'http://192.168.5.9/login': IP address used instead of domain
   - URL 'http://secure-login.paypa1.com/verify': Lookalike / suspicious keyword domain
   - Suspicious attachment type: invoice.exe
   - SPF authentication failed

From: hr@company.com  |  Score: 0  |  Verdict: LEGITIMATE
```

Program 30: Comprehensive Secure Email Analyzer

Aim: Develop a comprehensive secure email analyzer that classifies emails as legitimate, spam, or phishing by combining header analysis, content filtering, and machine learning techniques.

Requirements:

- Combine header checks (SPF/DKIM/domain match) with keyword and ML content analysis.
- Classify each email as LEGITIMATE, SPAM, SUSPICIOUS, or PHISHING.
- Demonstrate the analyzer on multiple sample emails.

Expected Learning Outcome: Understand how combining multiple detection layers improves overall email security classification.

```python
# Program 30: Comprehensive Secure Email Analyzer (Header + Content + ML)
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

# --- Train a small content classifier ---
train_texts = [
    "free lottery winner claim prize now",
    "urgent click here limited offer cash",
    "meeting agenda attached for review",
    "project timeline discussion tomorrow",
]
train_labels = [1, 1, 0, 0]
vectorizer = CountVectorizer().fit(train_texts)
clf = MultinomialNB().fit(vectorizer.transform(train_texts), train_labels)

SPAM_KEYWORDS = ["free", "winner", "urgent", "click here", "claim"]

def header_check(email):
    issues = []
    if email["spf"] != "pass":
        issues.append("SPF fail")
    if email["dkim"] != "pass":
        issues.append("DKIM fail")
    if email["from_domain"] != email["received_domain"]:
        issues.append("domain mismatch")
    return issues

def content_check(text):
    matched = [kw for kw in SPAM_KEYWORDS if kw in text.lower()]
    ml_pred = clf.predict(vectorizer.transform([text]))[0]
    return matched, ml_pred

def analyze(email):
    header_issues = header_check(email)
    matched_kw, ml_pred = content_check(email["body"])

    if header_issues and (matched_kw or ml_pred == 1):
        verdict = "PHISHING"
    elif ml_pred == 1 or len(matched_kw) >= 2:
        verdict = "SPAM"
    elif header_issues:
        verdict = "SUSPICIOUS"
    else:
        verdict = "LEGITIMATE"
    return verdict, header_issues, matched_kw, ml_pred

emails = [
    {"from_domain": "bank-alerts.com", "received_domain": "phishy-relay.ru", "spf": "fail",
        "dkim": "fail",
     "body": "Urgent! Click here to claim your free prize winner reward"},
    {"from_domain": "company.com", "received_domain": "company.com", "spf": "pass", "dkim":
        "pass",
     "body": "Please review the attached project timeline discussion notes"},
    {"from_domain": "shop.com", "received_domain": "shop.com", "spf": "pass", "dkim":
        "pass",
     "body": "Free shipping this weekend, click here for winner discounts"},
]

print("=== Comprehensive Secure Email Analyzer ===")
for email in emails:
    verdict, header_issues, matched_kw, ml_pred = analyze(email)
    print(f"\nFrom domain: {email['from_domain']}")
    print(f"  Header issues     : {header_issues if header_issues else 'none'}")
```